

Visual programming in C# (ICT Ed.465)

Operators in C#

Presented by:

Abishek Khawas

Symbol No.: 78214004

T.U. Regd. No.: 9-2-214-36-2021

What is operator?

An operator in programming is a symbol or keyword that performs a specific operation on one or more operands (variables, constants, or values). Operators are used to manipulate data and variables in a program.

- 1) Arithmetic Operators
- 2) Relational (Comparison) Operators
- 3) Logical Operators
- 4) Bitwise Operators
- 5) Assignment Operators
- 6) Unary Operators
- 7) Ternary (Conditional) Operator
- 8) Null-Coalescing Operators
- 9) Type Operators
- 10) Other Operators

1. Arithmetic Operators

Perform mathematical calculations:

- 1) $+$: Addition -> Adds two values (e.g., $a + b$, where $a = 5$ and $b = 3$, result is 8).
- 2) $-$: Subtraction -> Subtracts one value from another (e.g., $a - b$, result is 2).
- 3) $*$: Multiplication -> Multiplies two values (e.g., $a * b$, result is 15).
- 4) $/$: Division -> Divides one value by another (e.g., a / b , result is 1).
- 5) $\%$: Modulus -> Returns the remainder (e.g., $a \% b$, result is 2).

2. Relational (Comparison) Operators

Compare two values and return a boolean:

- 1) `==` : Equal to -> Checks if values are equal (e.g., `a == b`, result is false).
- 2) `!=` : Not equal to -> Checks if values are not equal (e.g., `a != b`, result is true).
- 3) `>` : Greater than -> Checks if left value is greater (e.g., `a > b`, result is true).
- 4) `<` : Less than -> Checks if left value is smaller (e.g., `a < b`, result is false).
- 5) `>=` : Greater than or equal to -> (e.g., `a >= b`, result is true).
- 6) `<=` : Less than or equal to -> (e.g., `a <= b`, result is false).

3. Logical Operators

Combine or negate conditions:

- 1) `&&` : Logical AND -> True if both conditions are true (e.g., `(a > 0 && b > 0)`, result is true).
- 2) `||` : Logical OR -> True if at least one condition is true (e.g., `(a > 0 || b < 0)`, result is true).
- 3) `!` : Logical NOT -> Negates the condition (e.g., `!(a > b)`, result is false).

4. Bitwise Operators

Work on bits:

- 1) $\&$: Bitwise AND -> Compares each bit and returns 1 if both bits are 1 (e.g., $a \& b$, where $a = 5$ and $b = 3$, result is 1).
- 2) $|$: Bitwise OR -> Compares each bit and returns 1 if either bit is 1 (e.g., $a | b$, result is 7).
- 3) $\wedge$: Bitwise XOR -> Returns 1 if bits are different (e.g., $a \wedge b$, result is 6).
- 4) $\sim$: Bitwise complement -> Flips the bits (e.g., $\sim a$, where $a = 5$, result is -6).
- 5) $\ll$: Left shift -> Shifts bits to the left (e.g., $a \ll 1$, result is 10).
- 6) $\gg$: Right shift -> Shifts bits to the right (e.g., $a \gg 1$, result is 2)

5. Assignment Operators

Assign or update values:

- 1) `=` : Assign -> Assigns a value (e.g., `a = 10`).
- 2) `+=` : Add and assign (e.g., `a += 5`, result is 15).
- 3) `-=` : Subtract and assign (e.g., `a -= 3`, result is 7).
- 4) `*=` : Multiply and assign (e.g., `a *= 2`, result is 20).
- 5) `/=` : Divide and assign (e.g., `a /= 5`, result is 2).
- 6) `%=` : Modulus and assign (e.g., `a %= 3`, result is 2).

6. Unary Operators

Operate on a single operand:

- 1) `+` : Unary plus -> Positive value (e.g., `+a`, where `a = 5`, result is 5).
- 2) `-` : Unary minus -> Negates the value (e.g., `-a`, result is -5).
- 3) `++` : Increment -> Increases value by 1 (e.g., `++a`, result is 6).
- 4) `--` : Decrement -> Decreases value by 1 (e.g., `--a`, result is 4).
- 5) `!` : Logical NOT -> Negates a boolean value (e.g., `!true`, result is false).

7. Ternary (Conditional) Operator

Short-hand for if-else:

1) `condition ? expr1 : expr2` -> If condition is true, return `expr1`; otherwise, `expr2`.

Example: `a > b ? "Yes" : "No"` -> If `a > b`, result is "Yes"; otherwise "No".

8. Null-Coalescing Operators

Handle null values:

- 1) ?? : Returns left value if not null; otherwise, right (e.g., a ?? b, where a = null, result is b).
- 2) ??= : Assigns the right value if the variable is null (e.g., a ??= 5, sets a to 5 if it's null).

9. Type Operators

Work with types:

- 1) `is` : Checks if an object is a specific type (e.g., `a is int`, result is true for int a).
- 2) `as` : Casts an object to a type or returns null (e.g., `obj as string`).
- 3) `typeof` : Returns the Type of a class or struct (e.g., `typeof(int)`).
- 4) `sizeof` : Gets the size of a type in bytes (e.g., `sizeof(int)` is 4).
- 5) `checked` : Ensures overflow raises an exception (e.g., `checked(a * b)`).
- 6) `unchecked` : Suppresses overflow exceptions (e.g., `unchecked(a * b)`).

10. Other Operators

- 1) `new` : Creates objects (e.g., `new int[5]` creates an array).
- 2) `sizeof` : Retrieves type size in bytes (used in unsafe context).
- 3) `=>` : Lambda operator for functions (e.g., `(x) => x * 2`).
- 4) `[]` : Access array elements (e.g., `arr[0]`).
- 5) `.` : Access members of an object (e.g., `obj.Property`).
- 6) `?:` : Null-conditional operator for safe access (e.g., `obj?.Property`).

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic design. The shapes are concentrated on the right side of the image, with some extending towards the left.

Thank You